

Implement Socket Communication to Simulate Data Acquisition in Cyber-Physical Systems

1 Preparation

1.1 Example CPS

In this project, we will use the IEEE 9-bus system as an example of CPS. The topology of the IEEE 9-bus system, i.e., the connection of substations through transmission lines, is shown in Figure 1.

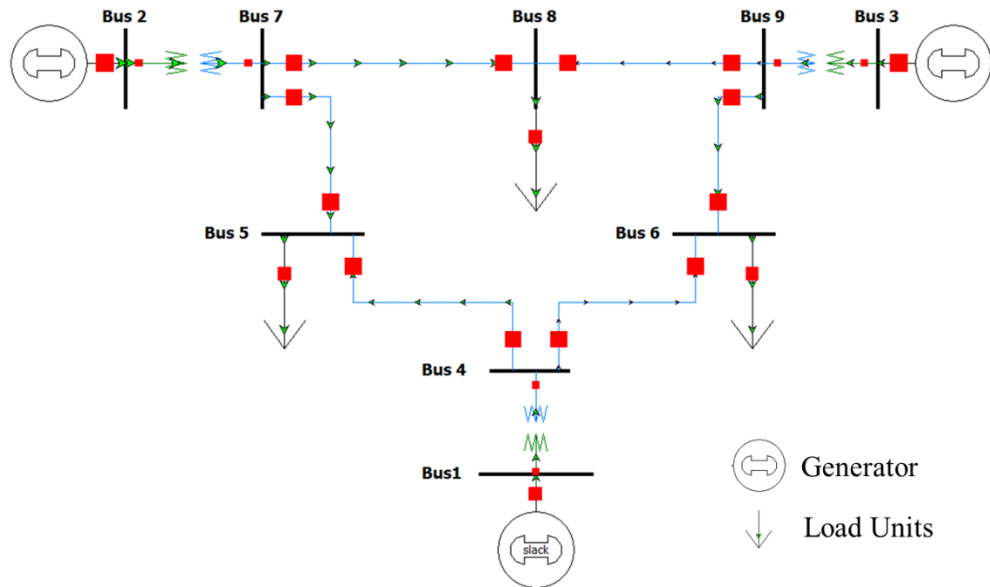


Figure 1. The topology of IEEE 9-bus system.

1.2 Simulation of communication networks

We still use Mininet to simulate a communication network with the topology shown in Figure 2. This network topology mimics but simplifies the communication infrastructure that can be found in today's cyber-physical systems. In this network, a central Control Center communicates with a Data Aggregator, which further communicates with Relay1, Relay2, Relay3, and Relay4. To simplify the project, we will fix the IP address for each host simulated in Mininet.

Communication networks used in CPS usually perform two operations: a control operation, which can change the configuration or states of physical processes, and polling operations, which periodically retrieve measurements indicating the state of physical processes. In power grids, each substation contains the following physical measurements: the magnitudes and phasor angles of voltage, real power injection, and reactive power injection. (Hint: power injection is calculated as differences between power generation and power consumption)

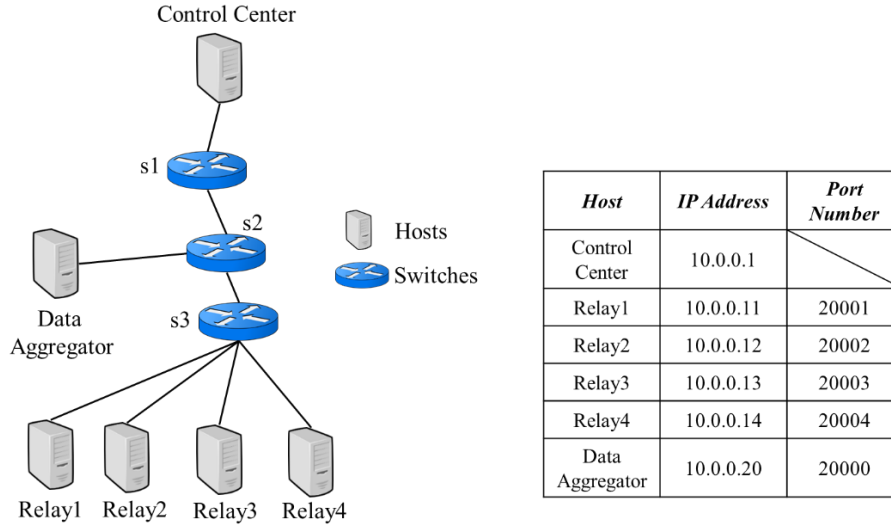


Figure 2. Network topology used in this project.

The measurements collected in each substation are shown in Table 1. To simplify the experiment, we remove the units of all measurements. We assign each measurement an index number, as shown in Table 1. Because there are only four relay machines, each relay will be responsible for multiple measurements. The assignment of the measurements is also specified in Table 1. Specifically, Relay 1 will be responsible for measurements from Bus 1 and Bus 2; Relay 2 will be responsible for measurements from Bus 3 and Bus 4; Relay 3 will be responsible for measurements from Bus 5 and Bus 6; and Relay 4 will be responsible for measurements from Bus 7, Bus 8, and Bus 9.

Table 1. Measurements collected by communication nodes.

Substation Index	Measurements			Communication Node
	Type	Index	Values	
Bus 1	Voltage magnitude	11	1.00	Relay 1
	Voltage phasor	12	0.00	
	Real power injection	13	71.95	
	Reactive power injection	14	24.07	
Bus 2	Voltage magnitude	21	1.00	
	Voltage phasor	22	9.67	
	Real power injection	23	163.00	
	Reactive power injection	24	14.46	
Bus 3	Voltage magnitude	31	1.00	Relay 2
	Voltage phasor	32	4.77	
	Real power injection	33	85.00	
	Reactive power injection	34	-3.65	
Bus 4	Voltage magnitude	41	0.99	
	Voltage phasor	42	-2.41	
	Real power injection	43	0.00	
	Reactive power injection	44	0.00	
Bus 5	Voltage magnitude	51	0.98	Relay 3
	Voltage phasor	52	-4.02	
	Real power injection	53	-90.00	

	Reactive power injection	54	-30.00	
Bus 6	Voltage magnitude	61	1.01	
	Voltage phasor	62	1.93	
	Real power injection	63	0.00	
	Reactive power injection	64	0.00	
Bus 7	Voltage magnitude	71	0.99	
	Voltage phasor	72	0.62	
	Real power injection	73	-100.00	
	Reactive power injection	74	-35.00	
Bus 8	Voltage magnitude	81	1.00	
	Voltage phasor	82	3.80	
	Real power injection	83	0.00	
	Reactive power injection	84	0.00	
Bus 9	Voltage magnitude	91	0.96	
	Voltage phasor	92	-4.35	
	Real power injection	93	0.00	
	Reactive power injection	94	0.00	

Relay 4

1.3 Network protocol

In this project, we use a protocol, based on which the Control Center collects measurements from Relays. We call this protocol DNP3m (“DNP3 minus”), as it represents a simplified version of the DNP3 protocol. It is an application layer protocol built on top of the TCP protocol.

The application layer structure is shown in Figure 3:

- **Request.** The “Request Header” contains two bytes. The first byte indicates that this is the request of the DNP3m protocol; the value of this byte should be set as “1” in a decimal. The second byte of the “Request Header” contains the length of this whole request message. Following the “Request Header,” the request contains indices of the measurements that the Control Center wants to collect. For example, if the request is issued from the Control Center to collect measurements of indices 11, 12, 13, 14, 21, 22, 23, and 24, the request should include all these indices following the “Request Header” with each index occupying one byte. **In this project, the Control Center collects all measurements listed in Table 1.**
- **Response.** The “Response Header” contains two bytes. The first byte indicates that this is the response of the DNP3m protocol; the value of this byte should be set as “11” in a decimal. The second byte of the “Response Header” contains the length of this whole response message. Following the “Response Header,” the response contains multiple “Measurement Tuples.” Each Measurement Tuple contains a Measurement Index occupying one byte and a Measurement Value occupying 4 bytes. In this tuple, the measurement index corresponds to the index included in the request, and the measurement value is the corresponding value shown in Table 1.

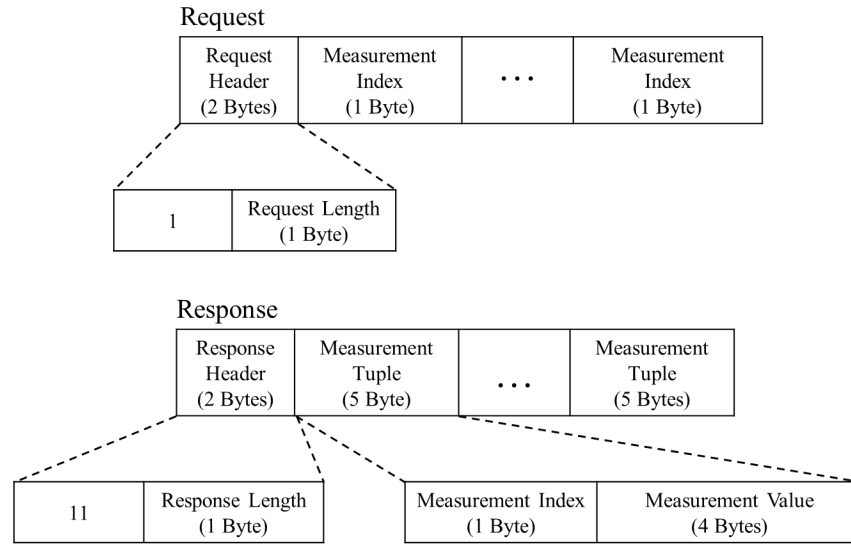


Figure 3. Application Layer Structure of the DNP3m Protocol.

2 Implementation

In this project, you will run a Control Center, Data Aggregator, and Relay applications, which are implemented in *control_center.py*, *data_aggregator.py*, *relay1.py*, *relay2.py*, *relay3.py*, and *relay4.py*. All these applications are implemented on top of a TCP socket. You should run these applications in the communication networks simulated in Mininet and thus simulate the communication as described in Section 1.2 (by running *build_net.py*). Please download the starter code at <https://github.com/hugolin615/ELE420520Spring2026/tree/main/project2>.

The implementation tasks of this project are located in *util.py*, which is used by the aforementioned applications to pack and unpack application layer payloads specified by the DNP3m protocol. The locations where you should input your codes are marked with #TODO. **You will need to read the implementations in all python scripts to make the implementations correct.**

The functionality of the Python scripts that I implemented for you is summarized as follows to help you.

- **util.py:** utility functions used by the following functions. Using "TODO" to locate your specific task.
 - Specifically, you will finish the implementation to pack and unpack DNP3m responses based on the comments. Please refer to the functions to pack and unpack DNP3m requests that I have implemented as references (*especially how to use "struct" module in Python*).
- *build_net.py*: this is the script to build a communication network in Mininet.
- *control_center.py*: this is implemented as a DNP3m master, so it is implemented as a TCP client. The Control Center only interacts with the Data Aggregator.
 - It will send a DNP3m request to ask for all measurements and expect a response from the Data Aggregator.

- When the control center receives a response from the Data Aggregator, it decodes or unpacks the byte stream according to the protocol specification shown in Figure 3. The Control Center application checks that the “Response Length” field contains an appropriate value.
- *relay1.py, relay2.py, relay3.py, and relay4.py*: these are implemented as DNP3m outstations, so they are implemented as four different TCP servers.
 - Each Relay only interacts with the Data Aggregator. When it receives a DNP3m request, it checks that the length field contains the correct value.
 - It first decodes the request and then responds by packing the measurements corresponding to the indices specified in the request in the application layer payload using the protocol specification shown in Figure 3.
- *data_aggregator.py*: this machine plays a similar role to the *Forward* machine you implemented in project 1. When the Data Aggregator receives a request from the Control Center, it decodes or unpacks the byte stream according to the protocol specification shown in Figure 3.
 - The Data Aggregator should read from the request to determine what indices are included.
 - The Data Aggregator uses Table 1 to put the indices into four different requests and send them to four Relays correspondingly.
 - The Data Aggregator waits until it receives four responses from four Relays. When the Data Aggregator receives four responses, it combines all measurement tuples into one response and sends the response to the Control Center.

To simplify the task, you should run your codes in the following order.

- [1] Enter Mininet CLI console by running your *build_net.py* (running commands “`sudo python ./build_net.py`” in the shell console). This will open seven Xterm consoles, one corresponding to the Control Center, two corresponding to the Data Aggregator, and four corresponding to Relay 1, Relay 2, Relay 3, and Relay 4. (*hint*: if you encounter some error messages when running this script, running “`sudo mn -c`” to clear up some resources left in your previous script running).
- [2] Execute the *relay1.py, relay2.py, relay3.py, and relay4.py* in the corresponding Xterm consoles (the order of executing these four scripts does not matter).
- [3] Execute the *data_aggregator.py* in the corresponding Xterm console. If your implementation is correct, it will connect to all four relay machines.
- [4] Execute the *control_center.py* in the corresponding Xterm console. If your implementation is correct, it will connect to the Data Aggregator machine and send the data.
- [5] To finish, type “exit” in the Mininet CLI console.

If your codes run correctly, you should see the *control_center.py* printing all measurements with their indices listed in Table 1.

3 Turn-in and Grading.

Please submit a single PDF file, named as “*Last Name_First Name_Project 2.pdf*,” including the following contents:

- **Python scripts (*util.py*) including your implementation (40%).**
- **A snapshot of running the Python scripts in Mininet (30%).** After running the scripts, take a snapshot to show the consoles of all machines. I suggested putting machines in the pattern shown in Figure 4 so that I can see the outputs. Please name this snapshot as *running_mininet.jpg* (or *running_mininet.png*).

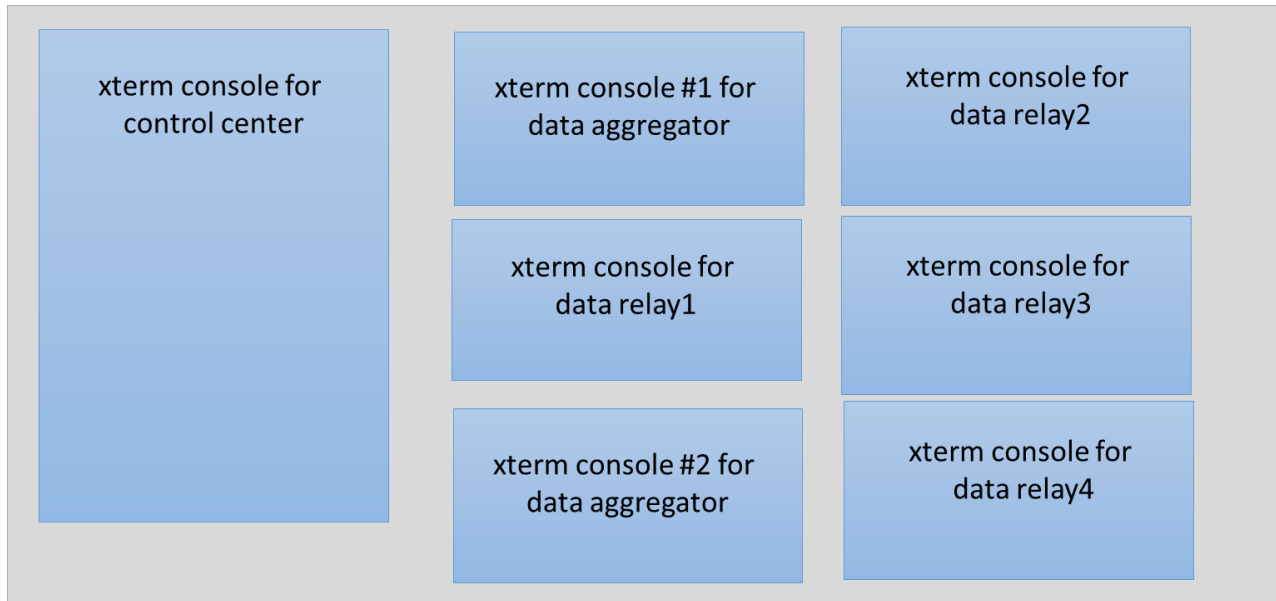


Figure 4. Suggested pattern to show all running machines in mininet.

Also submit a sperate Network trace file recorded by Wireshark (30%). In Mininet, in the Data Aggregator Xterm that is not running any script, you start Wireshark in that console. Using Wireshark to monitor the network traffic that will go through the Data Aggregator machine, while you run your Python scripts again (following Steps [1] to [5]). Save what you recorded as a network trace file (in the format of .pcapng or .pcap). Please name this trace file as *network_trace.pcap* or *network_trace.pcapng*.